

Forex AI Signal + Trading Bot

Complete Step-by-Step Beginner Setup Guide

Built for the project package: **forex_ai_signal_bot.zip**

Google Colab • Backtesting • Dashboard • Email Alerts • OANDA Practice/Live Path • MT4/MT5 Bridge

Use this guide like a recipe: start at Step 1 and continue one section at a time.

Risk warning: This guide is for education. Forex trading can lose money. The bot can help you trade, but it cannot guarantee profit. Start on demo.

Quick Table of Contents

1. What you are building
2. Full process map
3. Files inside the bot package
4. Accounts and websites you need
5. Download and prepare the project
6. Google Colab setup from zero
7. Run first AI Forex signal scan
8. Understand every signal column
9. Run backtest before trading
10. Dashboard setup in Colab
11. Free dashboard deployment with GitHub + Streamlit
12. Email signal alerts
13. Repeated live signal loop in Colab
14. OANDA practice auto-trading
15. Safety controls explained
16. News filter setup
17. Reading trade journal and risk state
18. OANDA live mode later
19. MT4/MT5 bridge setup
20. Troubleshooting
21. Final checklist

1. What You Are Building

You are building a beginner-friendly Forex AI signal and trading assistant. It scans Forex pairs, creates BUY/SELL/WAIT signals, shows signal strength, gives stop-loss and take-profit levels, shows active trading sessions, sends email alerts, and can optionally place trades through OANDA or MetaTrader.

The bot is made to run mainly in Google Colab, so you do not need to install Python on your own computer for the OANDA path.

Simple explanation: Google Colab is your free online Python computer. The bot runs there, creates signals, and can talk to online services like email and OANDA.

Important: A trading bot is not a guaranteed money machine. Your first goal is learning and safety, not fast profit.

2. Full Process Map

Follow this order. Do not jump to live trading first.

```
Download ZIP from workspace
↓
Upload folder to Google Drive
↓
Open FOREX_AI_BOT_COLAB.ipynb in Google Colab
↓
Install requirements inside Colab
↓
Run first signal scan → latest_signals.csv
↓
Run backtest.py → backtest_report.csv
↓
Open dashboard and learn signal table
↓
Add email alerts
↓
Connect OANDA practice account
↓
Demo auto-trading with safety controls
↓
Review trade_journal.csv and risk_state.json
↓
Improve settings and test for a long time
↓
Optional: small live trading only if demo is stable
```

Best beginner path

Use OANDA practice first because it works well from Colab through an online API. MetaTrader is optional and needs the MT4/MT5 terminal on a PC or VPS.

3. Files Inside the Bot Package

After you download and unzip forex_ai_signal_bot.zip, you will see a folder named forex_ai_signal_bot. These are the important files:

File/folder	What it does	When you use it
FOREX_AI_BOT_COLAB.ipynb	Main beginner notebook for Google Colab.	Start here.
forex_ai_bot.py	Main AI signal, email, OANDA execution, and safety engine.	Runs behind the scenes.
dashboard.py	Streamlit dashboard showing signals, charts, sessions, and journal.	When you want visual dashboard.
backtest.py	Simple historical train/test backtest.	Before demo/live trading.
run_colab_signal_loop.py	Repeats scan every chosen seconds.	For live signal loop in Colab.

File/folder	What it does	When you use it
api_bridge.py	Creates URL for MT4/MT5 EA to read latest signals.	Only for MetaTrader route.
requirements.txt	List of Python packages to install in Colab.	First setup.
.env.example	Template of settings/environment variables.	Reference only. Do not share secrets.
LIVE_TRADING_SAFETY_GUI DE.md	Extra safety details for broker execution.	Before any trading execution.
news_blackout.example.csv	Example file to avoid trading around news.	Optional safety.
metatrader/	MT4/MT5 EA files and MetaTrader setup guide.	Only for MT4/MT5.

4. Accounts and Websites You Need

Account/tool	Purpose	Required now?
Google account	Google Drive and Google Colab.	Yes
Google Colab	Run the bot without installing Python.	Yes
Google Drive	Store your bot folder and generated files.	Yes
Gmail App Password	Send email signal alerts.	Optional
OANDA practice account	Demo trading through API from Colab.	Recommended
GitHub	Store project for dashboard/API deployment.	Optional
Streamlit Community Cloud	Free dashboard deployment from GitHub.	Optional
Render/Railway/Fly/PythonAny where/Oracle VM	Possible API bridge hosting choices. Free tiers change.	Optional
MetaTrader 4 or 5	If you want EA trading with an MT broker.	Optional

For your current goal of using only Colab, focus on Google account, Colab, Drive, and OANDA practice account.

5. Download and Prepare the Project

- 1 In the workspace, download forex_ai_signal_bot.zip.
- 2 Also download Forex_AI_Bot_Complete_Beginner_Guide.pdf if you want this guide separately.
- 3 Right-click the ZIP file on your computer and extract/unzip it.
- 4 You should now see a folder named forex_ai_signal_bot.
- 5 Open Google Drive in your browser.
- 6 Upload the whole forex_ai_signal_bot folder into My Drive.
- 7 Wait until all files finish uploading.

How to confirm upload worked

Inside Google Drive, open forex_ai_signal_bot and check that these files exist:

- FOREX_AI_BOT_COLAB.ipynb
- forex_ai_bot.py
- requirements.txt
- dashboard.py

- backtest.py

6. Google Colab Setup From Zero

Step 6.1 — Open the notebook

- 1 Go to Google Drive.
- 2 Open the forex_ai_signal_bot folder.
- 3 Double-click FOREX_AI_BOT_COLAB.ipynb.
- 4 If Drive asks how to open it, choose Google Colaboratory.
- 5 If Google Colaboratory is missing, click Connect more apps and search Colaboratory.

Step 6.2 — Mount Google Drive

Run this cell in Colab. It connects Colab to your Google Drive so the bot can read the files.

```
from google.colab import drive
drive.mount('/content/drive')
```

Google will ask permission. Click the account, allow access, and wait until it says Drive mounted.

Step 6.3 — Move into the bot folder

```
%cd /content/drive/MyDrive/forex_ai_signal_bot
```

If you uploaded the folder inside another folder, change the path. Example:

```
%cd /content/drive/MyDrive/TradingBots/forex_ai_signal_bot
```

Step 6.4 — Check that files are present

```
!ls
```

You should see files like forex_ai_bot.py, dashboard.py, backtest.py, and requirements.txt. If not, your Colab is in the wrong folder.

Step 6.5 — Install required packages

```
!pip install -r requirements.txt
```

This installs packages inside Colab only. You still do not need to install Python on your laptop.

7. Run Your First AI Forex Signal Scan

Now run your first scan. This downloads data, creates indicators, trains a quick AI model, and creates signals.

```
from forex_ai_bot import scan_symbols

signals = scan_symbols(
    ['EURUSD', 'GBPUSD', 'USDJPY'],
    timeframe='1h',
    period='180d',
    output_path='latest_signals.csv'
)
```

After it finishes, run:

```
import pandas as pd
pd.read_csv('latest_signals.csv')
```

What success looks like

You should see a table with rows for EURUSD, GBPUSD, and USDJPY. The action may be BUY, SELL, or WAIT. WAIT is normal. The bot should not force trades all the time.

Good beginner pairs

Start with major pairs only:

- EURUSD
- GBPUSD
- USDJPY
- AUDUSD
- USDCAD
- USDCHF
- NZDUSD

Good beginner timeframe

Use 1h first. Very small timeframes like 1m and 5m can be noisy and may create too many signals for beginners.

8. Understand Every Signal Column

Column	Meaning	Beginner advice
symbol	The Forex pair.	Start with EURUSD, GBPUSD, USDJPY.
timeframe	Chart timeframe used.	Use 1h first.
action	BUY, SELL, WAIT, or ERROR.	Do not force trades when WAIT.
signal_strength_pct	How strong the signal is.	Consider 65%+ for execution testing.
hybrid_prob_up	AI + technical combined probability of up move.	Above threshold may create BUY.
ai_prob_up	Machine-learning model probability only.	Not a guarantee.
technical_score	EMA, MACD, RSI rule score.	Positive bullish, negative bearish.
last_price / entry	Current/estimated trade price.	Broker price may differ slightly.
stop_loss	Exit price if trade is wrong.	Never trade without SL.
take_profit	Target price if trade is right.	Helps avoid emotional exits.
risk_reward	Reward compared to risk.	Higher than 1 is generally better.
rsi	Momentum indicator.	Very high/low can warn of extremes.
model_accuracy	Recent test accuracy of model.	Use as clue, not guarantee.
session_quality	Liquidity quality: HIGH/GOOD/MEDIUM/LOW.	Avoid LOW as beginner.
reason	Short explanation of signal.	Read it before trusting signal.

Beginner rule: A signal is not permission to risk big money. It is only information. Safety settings must still approve the trade.

9. Run Backtest Before Trading

Backtesting checks the idea on past data. The included backtest is simple, not professional-grade, but it helps you avoid trading blindly.

```
!python backtest.py --symbols EURUSD,GBPUSD,USDJPY --period 180d --timeframe 1h --min-  
strength 0.62  
  
import pandas as pd  
pd.read_csv('backtest_report.csv')
```

Backtest result	Meaning	What to do
trades	Number of simulated trades.	Too few trades means not enough evidence.
win_rate	Percent of simulated wins.	High win rate alone is not enough.
avg_trade_return_pct	Average simulated return per trade.	Negative means weak settings.
total_return_pct	Total simplified return.	Negative needs improvement.
max_drawdown_pct	Worst simulated drop.	Large drawdown is dangerous.
model_accuracy	Direction accuracy.	Compare with baseline; do not overtrust.

If backtest is bad, do not trade. Try higher signal strength, fewer pairs, different timeframe, or improve the strategy.

10. Dashboard Setup in Colab

The dashboard gives you a visual page with best signal, signal table, gauges, sessions, chart, and journal.

Start Streamlit in Colab

```
!streamlit run dashboard.py --server.port 8501 > /content/streamlit.log 2>&1 &
```

Create a temporary public tunnel

```
!npm install -g localtunnel
!lt --port 8501
```

Colab will print a URL. Open it in a browser. This is for testing; it may stop when Colab disconnects.

If dashboard does not open

- Check that Streamlit is still running.
- Run the localtunnel command again.
- Check /content/streamlit.log for errors.
- Make sure you are inside the forex_ai_signal_bot folder.

11. Free Dashboard Deployment With GitHub + Streamlit

For a more stable dashboard page, deploy dashboard.py from GitHub to Streamlit Community Cloud.

Step 11.1 — Create GitHub repository

- 1 Create or log into GitHub.
- 2 Click New repository.
- 3 Name it forex-ai-signal-bot.
- 4 Choose Public if you want easiest free deployment. If private, check your deployment account limits.
- 5 Upload all files from forex_ai_signal_bot except private secret files like .env.

Step 11.2 — Deploy on Streamlit Community Cloud

- 1 Open Streamlit Community Cloud.
- 2 Sign in with GitHub.
- 3 Click New app.
- 4 Choose your GitHub repository.

- 5 Set branch to main.
- 6 Set main file path to dashboard.py.
- 7 Click Deploy.

Step 11.3 — Add safe environment settings

In Streamlit secrets/settings, add non-sensitive settings like SYMBOLS, TIMEFRAME, PERIOD, and MIN_SIGNAL_STRENGTH. Do not put broker live keys in a public dashboard unless you understand security.

```
SYMBOLS=EURUSD,GBPUSD,USDJPY
TIMEFRAME=1h
PERIOD=180d
MIN_SIGNAL_STRENGTH=0.62
```

12. Email Signal Alerts

Step 12.1 — Create Gmail App Password

- 1 Open your Google Account.
- 2 Go to Security.
- 3 Turn on 2-Step Verification.
- 4 Search for App Passwords.
- 5 Create an app password for Mail.
- 6 Copy the 16-character password.
- 7 Do not share it.

Step 12.2 — Set email variables in Colab

```
import os, getpass

os.environ['EMAIL_ENABLED'] = 'true'
os.environ['SMTP_HOST'] = 'smtp.gmail.com'
os.environ['SMTP_PORT'] = '465'
os.environ['SMTP_USER'] = input('Your Gmail: ')
os.environ['SMTP_PASSWORD'] = getpass.getpass('Gmail app password: ')
os.environ['EMAIL_TO'] = input('Send alerts to: ')
```

Step 12.3 — Run scan with email

```
from forex_ai_bot import scan_symbols
scan_symbols(['EURUSD', 'GBPUSD', 'USDJPY'], timeframe='1h', period='180d', email=True)
```

The bot usually sends email only when BUY/SELL appears. If all signals are WAIT, email may be skipped.

13. Repeated Live Signal Loop in Colab

To scan repeatedly every 15 minutes, run:

```
!python run_colab_signal_loop.py --loop --interval 900
```

900 seconds = 15 minutes. You can use 300 for 5 minutes, but do not scan too aggressively with free data.

Google Colab can disconnect. It is good for learning and testing, but not a guaranteed 24/7 server. For serious 24/5 live trading, later use VPS/cloud/broker hosting.

14. OANDA Practice Auto-Trading

This is the easiest execution path if you do not want to install Python locally. OANDA has a practice account and API that Colab can call.

Step 14.1 — Create practice account

- 1 Go to OANDA and create a practice/demo account.
- 2 Open account/API management area.
- 3 Generate a practice API token.
- 4 Copy your practice account ID.
- 5 Keep token private.

Step 14.2 — Set safe demo execution variables

```
import os, getpass

os.environ['TRADE_EXECUTION_ENABLED'] = 'true'
os.environ['AUTO_TRADE_MODE'] = 'auto'
os.environ['I_UNDERSTAND_TRADING_RISK'] = 'yes'
os.environ['OANDA_ENV'] = 'practice'
os.environ['OANDA_API_TOKEN'] = getpass.getpass('OANDA practice API token: ')
os.environ['OANDA_ACCOUNT_ID'] = input('OANDA practice account ID: ')

os.environ['OANDA_UNITS'] = '1000'
os.environ['MAX_UNITS'] = '1000'
os.environ['EXECUTION_MIN_SIGNAL_STRENGTH'] = '0.65'
os.environ['MAX_DAILY_LOSS_PCT'] = '2.0'
os.environ['MAX_TRADES_PER_DAY'] = '3'
os.environ['MAX_TRADES_PER_SYMBOL_PER_DAY'] = '1'
os.environ['MAX_OPEN_TRADES'] = '2'
os.environ['COOLDOWN_MINUTES'] = '60'
os.environ['MAX_SPREAD_PIPS'] = '2.5'
os.environ['CLOSE_OPPOSITE_POSITIONS'] = 'true'
```

Step 14.3 — Run demo execution

```
from forex_ai_bot import scan_symbols

scan_symbols(
    ['EURUSD', 'GBPUSD', 'USDJPY'],
    timeframe='1h',
    period='180d',
    execute=True
)
```

Meaning of execution result

Status	Meaning
sent	Order request was sent to OANDA. Check broker account.
blocked	Safety rule stopped the trade. This protects you.
skipped	Execution switch or risk confirmation is off.
error	Broker/API error occurred. Read message carefully.

15. Safety Controls Explained

The bot has safety controls because live trading without safety is dangerous.

Setting	Beginner value	Meaning
TRADE_EXECUTION_ENABLED	false until demo execution	Main broker execution switch.
AUTO_TRADE_MODE	signal_only first, then auto for demo	Must be auto before orders are placed.
I_UNDERSTAND_TRADING_RISK	yes only when ready	Confirms you understand risk.
EXECUTION_MIN_SIGNAL_STRENGTH	0.65 or higher	Blocks weak trades.
REQUIRE_STOP_LOSS	true	Blocks trades without stop loss.

Setting	Beginner value	Meaning
REQUIRE_TAKE_PROFIT	true	Blocks trades without take profit.
MAX_DAILY_LOSS_PCT	2.0	Stops if daily loss reaches 2%.
MAX_TRADES_PER_DAY	3	Prevents overtrading.
MAX_TRADES_PER_SYMBOL_PER_DAY	1	Limits repeated trades on one pair.
MAX_OPEN_TRADES	2	Limits simultaneous exposure.
COOLDOWN_MINUTES	60	Waits before another trade on same symbol.
MAX_SPREAD_PIPS	2.5	Avoids expensive spreads.
BLOCK_LOW_QUALITY_SESSIONS	true	Avoids poor liquidity sessions.

For beginners, do not reduce safety limits just to get more trades. Fewer, safer trades are better while learning.

16. News Filter Setup

High-impact news can make price move fast and unpredictably. The bot includes an optional news blackout file.

Create news_blackout.csv

In the project folder, create a file named news_blackout.csv like this:

```
time_utc,currency,impact,event
2026-06-23T12:30:00Z,USD,High,CPI release
2026-06-24T18:00:00Z,USD,High,FOMC rate decision
```

Enable news filter

```
import os
os.environ['NEWS_FILTER_ENABLED'] = 'true'
os.environ['NEWS_FILTER_STRICT'] = 'true'
os.environ['NEWS_BLACKOUT_FILE'] = 'news_blackout.csv'
os.environ['NEWS_BLACKOUT_MINUTES'] = '45'
os.environ['NEWS_BLOCK_IMPACTS'] = 'High'
```

This blocks trades for affected currency pairs from 45 minutes before to 45 minutes after the news time.

17. Reading Trade Journal and Risk State

Trade journal

```
import pandas as pd
pd.read_csv('trade_journal.csv')
```

Column	Meaning
timestamp_utc	When the execution decision happened.
status	sent, blocked, skipped, or error.
reason	Why the trade was blocked/skipped/error.
checks	Full safety checklist.
symbol/action	Pair and BUY/SELL.
signal_strength	Strength at execution time.
nav	Account NAV from broker if available.
daily_loss_pct	Daily loss percentage tracked by safety system.

Column	Meaning
spread_pips	Broker spread at execution check.
response	Broker response if an order was sent.

Risk state

risk_state.json stores daily starting NAV, peak NAV, and daily loss state. If a new UTC day starts, it resets daily baseline.

```
import json
print(open('risk_state.json').read())
```

18. OANDA Live Mode Later

Do not use live mode until demo results are stable and you understand every safety setting. Live trading can lose real money.

When you are ready in the future, OANDA live mode requires extra confirmation:

```
import os, getpass

os.environ['TRADE_EXECUTION_ENABLED'] = 'true'
os.environ['AUTO_TRADE_MODE'] = 'auto'
os.environ['LIVE_TRADING_ENABLED'] = 'true'
os.environ['LIVE_ACCOUNT_CONFIRMATION'] = 'I_ACCEPT_LIVE_RISK_2026'
os.environ['I_UNDERSTAND_TRADING_RISK'] = 'yes'
os.environ['OANDA_ENV'] = 'live'
os.environ['OANDA_API_TOKEN'] = getpass.getpass('OANDA LIVE token: ')
os.environ['OANDA_ACCOUNT_ID'] = input('OANDA LIVE account ID: ')
os.environ['OANDA_UNITS'] = '100'
os.environ['MAX_UNITS'] = '100'
```

Start tiny. Your first live goal is not profit; it is checking that everything works safely with real broker conditions.

19. MT4 / MT5 Bridge Setup

MetaTrader is optional. You only need this if you want the bot to trade through an MT4/MT5 broker. MetaTrader usually runs on a Windows PC or VPS.

How the bridge works

```
Python bot / API bridge creates signal:
EURUSD|BUY|0.68|1.08500|1.08000|1.09250|timestamp
↓
MT4/MT5 Expert Advisor reads the URL
↓
EA checks safety inputs
↓
EA places or closes trade in MetaTrader account
```

Bridge API setup

Run locally or deploy api_bridge.py online. Local command:

```
uvicorn api_bridge:app --host 0.0.0.0 --port 8000
```

Test URL:

```
http://127.0.0.1:8000/latest.txt?symbol=EURUSD
```

Cloud bridge deployment idea

- 1 Upload project to GitHub.
- 2 Create a web service on Render/Railway/Fly/PythonAnywhere or another host.
- 3 Build command: `pip install -r requirements.txt`
- 4 Start command: `uvicorn api_bridge:app --host 0.0.0.0 --port $PORT`

- 5 Add environment variable `API_SHARED_SECRET`.
- 6 Open `/scan?secret=YOUR_SECRET` to generate `latest_signals.csv`.
- 7 Point the EA to `/latest.txt?symbol=EURUSD&secret=YOUR_SECRET`.

MT5 EA install

- 1 Open MetaTrader 5.
- 2 Click File > Open Data Folder.
- 3 Open MQL5 > Experts.
- 4 Copy `MQL5_ForexAI_Bridge_EA.mq5` into that folder.
- 5 Open MetaEditor and compile it.
- 6 In MT5 go to Tools > Options > Expert Advisors.
- 7 Enable algorithmic trading.
- 8 Enable WebRequest and add your bridge base URL.
- 9 Attach EA to EURUSD chart.
- 10 Set SignalURL to your bridge latest.txt URL.
- 11 Keep `AllowNewTrades=false` first and watch logs.
- 12 Use demo before live.

MT4 EA install

Same process, but copy `MQL4_ForexAI_Bridge_EA.mq4` into MQL4 > Experts.

EA safety inputs

EA input	Meaning
<code>AllowNewTrades</code>	Master switch. False means signals only, no new trades.
<code>Lots</code>	Trade size. Use 0.01 demo first.
<code>MinimumStrength</code>	Ignore weak signals.
<code>MaxSpreadPoints</code>	Avoid high spread.
<code>MaxTradesPerDay</code>	EA daily overtrading control.
<code>MaxOpenPositions</code>	Limits open EA trades.
<code>CooldownMinutes</code>	Wait time before next trade.
<code>MaxDailyLossPercent</code>	EA-side daily equity loss stop.
<code>MagicNumber</code>	EA trade ID so it manages only its own trades.

20. Troubleshooting

Problem	Likely cause	Fix
Colab says file not found	Wrong folder path.	Run <code>!ls</code> and <code>cd</code> into <code>forex_ai_signal_bot</code> .
No module named...	Packages not installed.	Run <code>!pip install -r requirements.txt</code> .
No data returned	Free data issue or symbol issue.	Try EURUSD, 1h, 180d.

Problem	Likely cause	Fix
Signal says ERROR	Data or model problem.	Read reason column. Try another pair/timeframe.
Email fails	Wrong Gmail setup.	Use Gmail App Password, not normal password.
OANDA unauthorized	Wrong token/account/environment.	Check practice vs live and account ID.
Trade blocked	Safety rule failed.	Read trade_journal.csv reason and checks.
Dashboard not opening	Tunnel or Streamlit stopped.	Restart Streamlit and localtunnel.
Colab disconnected	Colab runtime limit.	Restart. For 24/5 use VPS/cloud later.
MT WebRequest failed	URL not allowed.	Add bridge base URL in Expert Advisors settings.
EA receives signal but no trade	AllowNewTrades false or safety block.	Check EA inputs and journal/logs.

21. Final Beginner Checklist

Before first signal scan

- ☐ I downloaded forex_ai_signal_bot.zip.
- ☐ I uploaded forex_ai_signal_bot folder to Google Drive.
- ☐ I opened FOREX_AI_BOT_COLAB.ipynb in Colab.
- ☐ I mounted Google Drive.
- ☐ I changed directory into the correct folder.
- ☐ I installed requirements.txt.

Before demo auto-trading

- ☐ I ran first signal scan successfully.
- ☐ I viewed latest_signals.csv.
- ☐ I ran backtest.py.
- ☐ I understand BUY, SELL, WAIT.
- ☐ I set OANDA_ENV=practice.
- ☐ I set AUTO_TRADE_MODE=auto only for demo.
- ☐ I set daily loss, max trades, cooldown, and spread filter.

Before any real live trading

- ☐ I demo-tested for a meaningful period.
- ☐ I reviewed trade_journal.csv.
- ☐ I understand the bot can lose money.
- ☐ I know how to stop execution immediately.
- ☐ I will start tiny only if I personally accept the risk.
- ☐ I will not remove stop loss or safety limits.

Final rule: If you do not understand a setup step, stop and ask for help before continuing. Never guess with live money.

End of detailed setup guide.